Controlling NUMA Memory Placement

Lecture notes — HPC 1 — Università di Trieste, A.Y. 2025/2026
Luca Tornatore, luca.tornatore@inaf.it

Controlling NUMA Memory Placement
- The problem
- 1. The command-line tool: `numactl`
  - Querying the system
  - Memory placement policies
  - CPU placement
  - Combining CPU and memory policies
  - Important: `numactl` sets the *default* policy for the *whole* process
- 2. The C library: `libnuma`
  - Initialisation
  - Querying the topology
  - Allocating on a specific node
  - Allocating with interleaving
  - Setting a thread's default policy
  - Running on a specific node
  - Strict vs. preferred
- 3. A practical example: per-socket arrays in OpenMP
- 4. The lower-level interface: `mbind` and `set_mempolicy`
- 5. Diagnostic tools
- 6. Reading `/proc/self/numa_maps`
  - A real example
  - How to read it field by field
  - What to look for
  - Reading it from inside your own code
  - Page size
- 7. Practical advices

The problem

On a multi-socket machine, where your data physically lives in DRAM matters. “Local” memory (attached to the socket where your thread is running) is faster than “remote” memory (attached to a different socket, reachable only through UPI/QPI links). The difference can easily be 2×in bandwidth and significantly more in latency (the `numactl -h` goves you the memory distance matrix in terms of latency).

We mentioned that by default, Linux uses a "first-touch" policy: a page of virtual memory gets allocated on the NUMA node where the thread that first writes to it is running. This is usually a sensible default, but it can go wrong in several ways:

- The thread that initialises the data is not the thread that will use it.
- The kernel's automatic NUMA balancer (`numa_balancing`) migrates pages silently (no messages are issued).

- The allocator reuses previously freed virtual address ranges, and the physical pages end up wherever they were last time.

None of these require root provileges to be fixed. The `numactl` command and the `libnuma` library give you control over memory placement policy, and they work with regular user's privileges.

---

1. The command-line tool: `numactl`

`numactl` wraps your program and sets a NUMA policy *before* the program starts. The policy is inherited by all child processes and threads. No root required — you're only controlling your own process's memory and CPU placement.

Querying the system

Check what you have before deciding your policies:

```
# Show your current NUMA policy
numactl --show

# Show the hardware: nodes, CPUs, memory, distances
numactl --hardware
```

The `--hardware` (shortcut_ `-H`) output includes the NUMA distance matrix — the relative cost of accessing memory from one node to another.
Values of 10 is assigned to "local" memory; higher values to "remote" memory: the larger the value, the larger the latency cost.

Memory placement policies

There are four main policies you can set from the command line. They control where *future* memory allocations (via `mmap`, `malloc`, etc.) will be physically placed.

`--localalloc` — allocate on the node where the thread is currently running. This is close to the default first-touch policy, but stated explicitly. Useful as a baseline.

```
numactl --localalloc ./my_program
```

`--membind=nodes` — hard-bind all allocations to specific nodes. If those nodes run out of memory, allocation *fails* (returns NULL or triggers OOM) rather than falling back to another node.

```
# All memory goes on node 0
numactl --membind=0 ./my_program

# Memory can go on nodes 0 or 2
numactl --membind=0,2 ./my_program
```

`--preferred=node` — try to allocate on this node first; if it's full, fall back to other nodes. Less strict than `--membind`.

```
numactl --preferred=1 ./my_program
```

`--interleave=nodes` — round-robin page allocation across the listed nodes. Each successive page goes on the next node in the list. This gives up locality in exchange for even bandwidth distribution — useful when you have no idea which thread will access which pages, or when you want to maximise aggregate bandwidth across all memory controllers (example: sorting).

```
# Interleave across all nodes
numactl --interleave=all ./my_program

# Interleave across nodes 0 and 1 only
numactl --interleave=0,1 ./my_program
```

CPU placement

You can also control where your threads run. This is separate from memory policy but obviously related, since you want threads and their data on the same node.

`--cpunodebind=nodes` — restrict execution to the CPUs on the specified nodes.

```
# Run only on CPUs belonging to node 0
numactl --cpunodebind=0 ./my_program
```

`--physcpubind=cpus` — pin to specific physical CPU numbers.

```
# Pin to CPUs 0-11 (whatever node they're on)
numactl --physcpubind=0-11 ./my_program
```

Combining CPU and memory policies

A common pattern is binding both threads and memory to the same node:

```
numactl --cpunodebind=0 --membind=0 ./my_program
```

Or bind threads to a node but let memory spread to maximize the aggregated bandwidth or sparse memory access:

```
numactl --cpunodebind=0 --interleave=all ./my_program
```

Important: `numactl` sets the *default* policy for the *whole* process

The policy applies to all memory allocations that don't have a more specific policy. If your code uses OpenMP with threads on multiple sockets, `numactl --membind=0` will force *all* memory onto node 0 — even allocations made by threads on node 1. That's probably not what you want for an OpenMP code.
For per-thread control, you need the library (see Section 2).

For a typical use case: `numactl --localalloc` combined with proper thread pinning (e.g., `OMP_PLACES=sockets OMP_PROC_BIND=spread`) and correct first-touch initialisation is usually enough. **The `--localalloc` makes the first-touch policy explicit *and* disables the kernel's automatic page migration.**

---

2. The C library: `libnuma`

When you need per-allocation control — "put *this* array on node 2, put *that* array on node 0" — you need to use the `libnuma` library, the C library that ships with the `numactl` package.

Compile and link with:

```
gcc -o my_program my_program.c -lnuma
```

Header: `#include <numa.h>`.
For the lower-level system call wrappers: `#include <numaif.h>`.

Initialisation

Good practice: always call `numa_available()` first. It returns -1 if NUMA support is not available.

```
#include <numa.h>

if (numa_available() == -1) {
    fprintf(stderr, "NUMA not available on this system\n");
    exit(1);
}
```

Querying the topology

```
int nnodes = numa_max_node() + 1;         // number of NUMA nodes
int ncpus  = numa_num_configured_cpus();  // number of CPUs in the system
```

Allocating on a specific node

The simplest and most useful function:

```
// Allocate 'size' bytes on NUMA node 'node'
void *buf = numa_alloc_onnode(size, node);

// When done:
numa_free(buf, size);
```

Under the hood, this does `mmap` + `mbind` — it allocates virtual pages and then binds them to the specified node. The physical pages are allocated on first touch, but the `mbind` policy ensures they go on the right node regardless of which thread touches them first.

NOTE: You must free with `numa_free()`, not `free()`.

Allocating with interleaving

```
// Allocate 'size' bytes interleaved across all nodes
void *buf = numa_alloc_interleaved(size);

// Or interleave across a specific set of nodes:
struct bitmask *nodes = numa_allocate_nodemask();
numa_bitmask_setbit(nodes, 0);
numa_bitmask_setbit(nodes, 2);
void *buf = numa_alloc_interleaved_subset(size, nodes);
numa_bitmask_free(nodes);

numa_free(buf, size);
```

Setting a thread's default policy

You can change the memory policy for the calling thread, affecting all subsequent allocations:

```
// All future allocations on this thread go to node 2
numa_set_preferred(2);

// Or: all future allocations interleaved across all nodes
numa_set_interleave_mask(numa_all_nodes_ptr);

// Or: bind to specific nodes (strict — fail if not available)
struct bitmask *nodes = numa_allocate_nodemask();
numa_bitmask_setbit(nodes, 0);
numa_set_membind(nodes);
numa_bitmask_free(nodes);
```

These calls affect the calling thread only. In an OpenMP context, each thread can set its own policy independently:

```
#pragma omp parallel
{
    int mynode = cyclic_mapping(omp_get_thread_num());  // your mapping
    numa_set_preferred(mynode);

    // allocations from this point on prefer 'mynode'
    double *local_buf = malloc(N * sizeof(double));
    // ...
}
```

Running on a specific node

```
// Bind the calling thread to all CPUs on node 'node'
numa_run_on_node(node);

// Bind both CPU and memory:
struct bitmask *nodemask = numa_allocate_nodemask();
numa_bitmask_setbit(nodemask, node);
numa_bind(nodemask);   // sets both CPU affinity and membind
numa_bitmask_free(nodemask);
```

Strict vs. preferred

By default, `numa_alloc_onnode()` and `numa_set_membind()` will fall back to other nodes if the requested one is full.
To make them fail instead you can call:

```
numa_set_strict(1);   // from now on, no fallback — fail if node is full
```

---

3. A practical example: per-socket arrays in OpenMP

Here's the pattern from the bandwidth benchmark we discussed in class, but using `libnuma` instead of relying on first-touch (see the document `lecture_notes---openmp_memory_allocation` for a guided tour on all the features):

```
#include <omp.h>
#include <numa.h>

#define ARRAY_SIZE (1 << 26)  // 512 MB of doubles

int nsockets;
double *data[MAX_SOCKETS];

#pragma omp parallel num_threads(nsockets) proc_bind(spread)
{
    int me       = omp_get_thread_num();
    int mysocket = omp_get_place_num();

    // Allocate explicitly on this socket's NUMA node
    data[me] = (double *)numa_alloc_onnode(
                    ARRAY_SIZE * sizeof(double), mysocket);

    if (data[me] == NULL) {
        fprintf(stderr, "numa_alloc_onnode failed on node %d\n", mysocket);
        exit(1);
    }

    // Initialise — first-touch is now redundant, but we do it
    // anyway because we need to fill the array with values
```

```
    for (int i = 0; i < ARRAY_SIZE; i++)
        data[me][i] = (double)i;

    #pragma omp barrier

    // ... benchmark code ...

    #pragma omp barrier
    numa_free(data[me], ARRAY_SIZE * sizeof(double));
}
```

With `numa_alloc_onnode`, the physical pages are bound to the specified node by `mbind` *before* any touch. It doesn't matter which thread initialises them — they stay on the right node.
This is immune to NUMA balancing, to allocator reuse, and to any other kernel heuristic. Compare with the original version that used `aligned_alloc` + first-touch: that version produced wildly asymmetric bandwidth numbers when `numa_balancing` was on, because the kernel was migrating pages between the allocation and the measurement.
Switching to `numa_alloc_onnode` (or turning off `numa_balancing`) fixed it.

---

4. The lower-level interface: `mbind` and `set_mempolicy`

`libnuma` is a wrapper around two Linux system calls (you rarely need to use them directly, but it's good to know they exist).

`set_mempolicy(mode, nodemask, maxnode)` — sets the *thread's default* memory policy.
All subsequent `mmap`/`malloc` calls inherit this policy. This is what `numa_set_preferred()`, `numa_set_membind()`, and `numa_set_interleave_mask()` call internally.

`mbind(addr, len, mode, nodemask, maxnode, flags)` — sets the memory policy for a *specific range* of virtual addresses. This is what `numa_alloc_onnode()` calls after `mmap`.
The `flags` argument can include `MPOL_MF_MOVE`, which will physically migrate pages that are already faulted in — useful if you want to re-place data that was already allocated somewhere else.

Both are defined in `<numaif.h>` and do not require root privileges.

---

5. Diagnostic tools

A small list of useful command-line tools to explore the topology of the node:

- **`numactl --show`** — displays the current process's NUMA policy and allowed nodes.
- **`numactl --hardware`** — (`-H` is the shorcut) shows nodes, CPUs, memory, and the distance matrix.
- **`numastat`** — per-node allocation statistics. With `-p <pid>`, shows where a specific process's pages are physically located.

```
# While the program is running in another terminal:
numastat -p $(pgrep my_program)
```

- **`/proc/self/numa_maps`** — per-VMA (virtual memory area) breakdown showing which node each

memory region's pages are on. You can read this from inside your own program:

```
// Quick and dirty — dump your own NUMA placement
char cmd[64];
snprintf(cmd, sizeof(cmd), "cat /proc/%d/numa_maps", getpid());
system(cmd);
```

---

6. Reading `/proc/self/numa_maps`

This file is your ground truth for page placement. Every memory region (VMA) in your process gets one line. We use this in the `test_bandwidth.*.c` suite of tests to probe that our pages are on different numa nodes.

A real example

Taken from the output of `test_bandwidth.probe_allocation.c`

```
7f1a3c000000 bind:0 anon=131072 dirty=131072 active=0 N0=131072
7f1a5c600000 bind:2 anon=131072 dirty=131072 active=0 N2=131072
7f1a80000000 default file=/lib/x86_64-linux-gnu/libc-2.31.so mapped=474 N0=474
7f1a80200000 default file=/lib/x86_64-linux-gnu/libm-2.31.so mapped=163 N0=92 N1=71
7ffe4a600000 default stack anon=12 dirty=12 active=4 N0=12
55a3c7800000 default heap anon=48 dirty=48 active=0 N0=31 N1=17
```

How to read it field by field

First field — virtual address. The starting address of the memory region. You can cross-reference this with `/proc/self/maps` to see the full extent of the region, its permissions, and what file (if any) backs it.

Second field — effective memory policy. The policy the kernel is actually using for this region. Possible values:

Policy	Meaning
`default`	The process-wide default (usually local allocation)
`prefer:N`	Prefer node N, fall back to others if N is full
`bind:N` or `bind:N0,N1`	Hard-bound to these nodes; fail if they're full
`interleave:N0,N1,...`	Round-robin across listed nodes

The "effective" policy is what's actually in effect, not necessarily what the process explicitly set. If you set `default` for a range, the effective policy is whatever the thread's default policy is — which might be `prefer:0` if the process was launched with `numactl --preferred=0`.

Remaining fields — page statistics. These appear as `key=value` pairs, in no fixed order:

Field	Meaning
`N0=1234`	1234 pages physically on NUMA node 0

Field	Meaning
`N1=567`	567 pages on node 1
`anon=N`	N anonymous pages (heap, stack, mmap without a file)
`dirty=N`	N pages that have been written to
`mapped=N`	N pages currently mapped into the process
`active=N`	N pages on the kernel's active list (recently accessed)
`mapmax=N`	Max sharing count — how many processes map the most-shared page
`swapcache=N`	N pages with swap entries (have been swapped at some point)
`writeback=N`	N pages currently being written to disk
`file=...`	The backing file, if any (shared library, mmap'd file)
`heap`	This region is the process heap
`stack`	This region is a thread stack
`huge`	Huge pages; counts are in huge-page units, not 4K pages

The `N0`, `N1`, ... fields are the ones you care about most. They tell you where each page *physically* lives right now. The sum of all `N<x>` values for a line should equal the total number of mapped pages in that region.

What to look for

Checking that first-touch worked. Suppose thread 0 on node 0 allocated and initialised a 512 MB array. You expect all its pages on node 0. Look for the corresponding VMA (large `anon` count at the right address range) and verify that only `N0` appears:

```
7f1a3c000000 default anon=131072 dirty=131072 N0=131072
```

131072 pages × 4096 bytes = 512 MB, all on N0. Good.

Spotting NUMA balancer migration. If the same array shows:

```
7f1a3c000000 default anon=131072 dirty=131072 N0=98000 N1=20000 N2=8072 N3=5000
```

then 33,072 pages have been migrated away from node 0. That's ~25% of the array sitting on remote nodes. This is what `numa_balancing` does when it thinks another node's threads are using those pages more.

Checking explicit placement. If you used `numa_alloc_onnode(size, 2)`, you expect:

```
7f1a5c600000 bind:2 anon=131072 dirty=131072 N2=131072
```

The policy says `bind:2` (not `default`), and all pages are on N2.

Shared library pages. Libraries like libc are typically mapped read-only and shared. Their pages end up wherever they were first faulted in — often scattered:

```
7f1a80200000 default file=/lib/x86_64-linux-gnu/libm-2.31.so mapped=163 N0=92 N1=71
```

This is normally harmless. Library text pages are read-only, so there's no coherence traffic.
The latency difference for fetching instructions from a remote node vs. local is real but usually negligible compared to data access patterns.

Heap pages split across nodes. If you see:

```
55a3c7800000 default heap anon=48 dirty=48 N0=31 N1=17
```

that means the heap has pages on two nodes. If you expected all heap on node 0, something is off — either the thread migrated mid-allocation, or the NUMA balancer moved pages, or another thread allocated heap memory while running on node 1.

Reading it from inside your own code

```c
#include <stdio.h>
#include <unistd.h>

void dump_numa_maps(void) {
    char path[64];
    snprintf(path, sizeof(path), "/proc/%d/numa_maps", getpid());
    FILE *f = fopen(path, "r");
    if (!f) return;
    char line[512];
    while (fgets(line, sizeof(line), f))
        fputs(line, stderr);
    fclose(f);
}
```

Call this after allocation and initialisation, before the hot loop, to verify placement. In an OpenMP code, call it from a single thread (inside `#pragma omp single` or from thread 0) to avoid garbled output.

Page size

All page counts in `numa_maps` are in units of the system page size — usually 4096 bytes on x86-64. So `N0=131072` means 131072 × 4096 = 536,870,912 bytes = 512 MB. If the region uses huge pages (2 MB), the line will show `huge` and the counts are in huge-page units instead.

---

7. Practical advices

For OpenMP codes on multi-socket nodes:

1. Pin threads: `OMP_PLACES=sockets` (or `cores`), `OMP_PROC_BIND=spread` (or `close`).
2. Initialise data in parallel, with each thread touching the data it will use (first-touch).

3. If first-touch is not practical (complex initialisation, data shared across threads), use `numa_alloc_onnode()` to place data explicitly.
4. If you suspect, or probe, that the NUMA balancer is interfering, use `mbind`-based allocation and verify with `numastat -p` (or from within your code).

For MPI codes: each rank is a separate process and typically uses only local memory. NUMA is less of an issue unless you're running multiple ranks per socket (hybrid MPI+OpenMP). In that case, the same advice applies within each node.

For benchmarks: verify page placement with `numastat -p` or `/proc/self/numa_maps`. First-touch is a *policy*, not a guarantee — the kernel can migrate pages afterwards if `numa_balancing` is active. Our bandwidth benchmark produced completely misleading results until we either disabled balancing or used explicit `numa_alloc_onnode`.

Cross-references: for NUMA topology and inter-socket coherence cost, see the lecture notes on OpenMP performance (the platform comparison in Section 5). For the MESI protocol and cache-line bouncing, see Intro to Parallelism slides 81–85.